

# VegeLink Ghana — Frontend Technical Specification

**Version:** 1.0  
**Date:** June 28, 2026  
**Author:** Frontend Team  
**Audience:** Frontend dev, backend team, challenge judges  
**Status:** Active — challenge period Jun 29-Jul 10

---

## 1. Overview

VegeLink Ghana is a farmer-to-buyer digital marketplace for perishable vegetables. The frontend is a shared React codebase targeting both web (React.js) and mobile (React Native), with maximum code reuse across hooks, API calls, state management, and validation logic.

### 1.1 Platform targets

| Target       | Technology                            | Deployment                 |
|--------------|---------------------------------------|----------------------------|
| Web app      | React.js 18 + Vite                    | Vercel                     |
| Mobile app   | React Native 0.74 + Expo              | Expo Go (demo) / EAS Build |
| Shared logic | Custom hooks, Zod schemas, API client | Imported by both           |

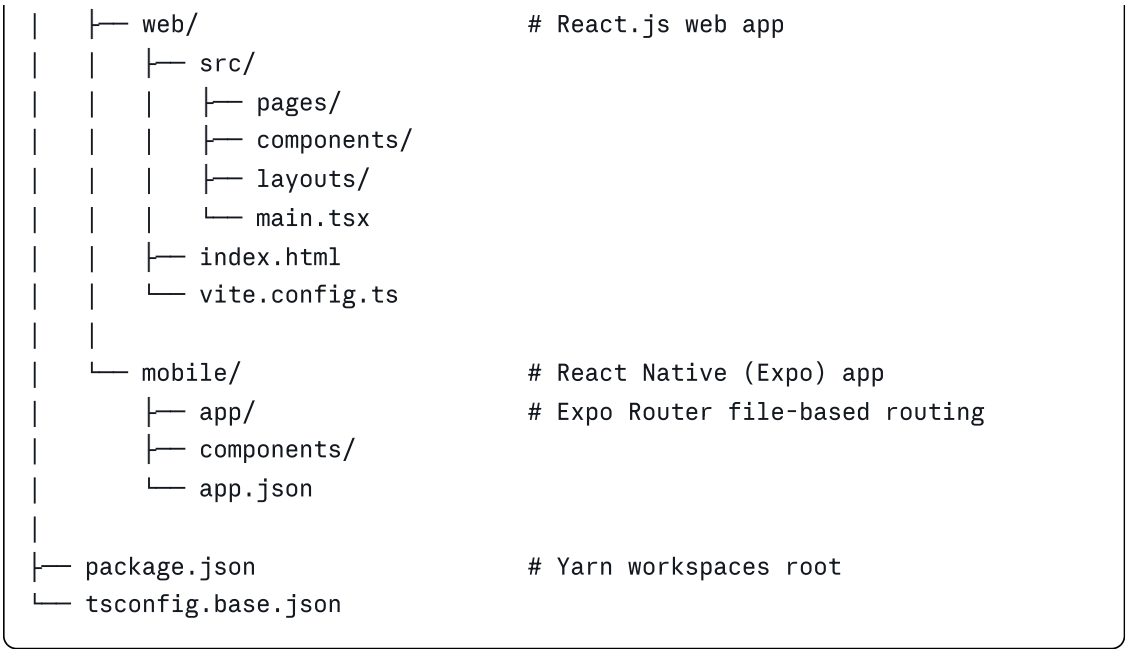
### 1.2 Design principles

- **Offline-aware:** Show cached data and graceful empty states when network is unavailable. Mutations queue and retry.
- **Low-bandwidth first:** Cloudinary thumbnail URLs for list views; full images only on detail screens.
- **Role-gated UI:** The same codebase renders different navigation and screens depending on `user.role` (`farmer`, `buyer`, `transporter`, `agent`).

- **SMS-aware flows:** Many confirmation steps require no in-app action (handled via SMS by the backend). The UI must reflect pending states clearly without blocking the user.
- 

## 2. Repository structure

```
vegeliink-frontend/  
├─ packages/  
│   ├─ shared/                # Platform-agnostic shared logic  
│   │   └─ api/               # Typed API client (axios)  
│   │       └─ client.ts      # Base axios instance, interceptors  
│   │       └─ auth.api.ts  
│   │       └─ listings.api.ts  
│   │       └─ orders.api.ts  
│   │       └─ transport.api.ts  
│   │       └─ payments.api.ts  
│   │       └─ messages.api.ts  
│   │   └─ hooks/             # React hooks (usable in web + RN)  
│   │       └─ useAuth.ts  
│   │       └─ useListings.ts  
│   │       └─ useOrders.ts  
│   │       └─ useTransport.ts  
│   │       └─ useGeolocation.ts  
│   │       └─ useRealtimeStatus.ts  
│   │   └─ stores/             # Zustand global state  
│   │       └─ auth.store.ts  
│   │       └─ cart.store.ts  
│   │       └─ notification.store.ts  
│   │   └─ schemas/           # Zod validation schemas (shared with  
backend logic)  
│   │       └─ auth.schemas.ts  
│   │       └─ listing.schemas.ts  
│   │       └─ order.schemas.ts  
│   │   └─ types/             # Shared TypeScript types  
│   │       └─ index.ts  
│   │  
│   │
```



### 3. Tech stack

| Concern          | Choice                   | Reason                                                                    |
|------------------|--------------------------|---------------------------------------------------------------------------|
| Web framework    | React 18 + Vite          | Fast HMR, ES modules, TypeScript first-class                              |
| Mobile framework | React Native 0.74 + Expo | Shared JS logic with web; Expo simplifies camera, location, notifications |
| Routing (web)    | React Router v6          | Industry standard, nested layouts                                         |
| Routing (mobile) | Expo Router              | File-based, deep-linking, tab navigation                                  |
| State management | Zustand                  | Lightweight, no boilerplate, works in RN without config                   |

| Concern          | Choice                                              | Reason                                                               |
|------------------|-----------------------------------------------------|----------------------------------------------------------------------|
| Server state     | TanStack Query (React Query)                        | Caching, background refetch, optimistic updates                      |
| Forms            | React Hook Form + Zod                               | Same Zod schemas shared with backend validation                      |
| HTTP client      | Axios                                               | Interceptors for JWT injection + token refresh                       |
| Maps (web)       | Leaflet.js + react-leaflet                          | Free, no billing, sufficient for MVP                                 |
| Maps (mobile)    | react-native-maps                                   | Native map with marker clustering                                    |
| Image upload     | react-dropzone (web) / expo-image-picker (mobile)   | Platform-appropriate; uploads to <code>/listings/upload-image</code> |
| Real-time        | Socket.io client                                    | Delivery status updates on transport tracking screen                 |
| Notifications    | react-hot-toast (web) / expo-notifications (mobile) | In-app toasts + push notifications                                   |
| Styling (web)    | Tailwind CSS                                        | Utility-first, responsive, no runtime overhead                       |
| Styling (mobile) | StyleSheet + NativeWind                             | NativeWind brings Tailwind class syntax to RN                        |
| Language         | TypeScript 5                                        | Shared types between web and mobile via <code>packages/shared</code> |

## 4. Authentication flow

### 4.1 Token management

The backend issues:

- **Access token** — 15-minute JWT, returned in response body
- **Refresh token** — 7-day JWT, set as `(httpOnly)` cookie (web) or stored in `(expo-secure-store)` (mobile)

```
| Registration (2-step) |  
|  
| Step 1: POST /auth/register → OTP sent via Arkesel SMS |  
| Step 2: POST /auth/verify-otp → access_token + cookie |  
|                               + user object |
```

```
| Token refresh (axios interceptor) |  
|  
| Request fails with 401 |  
|   → POST /auth/refresh (cookie sent automatically) |  
|   → New access_token received |  
|   → Original request retried with new token |  
|   → If refresh also 401: clear auth state, redirect login |
```

#### 4.2 Auth store (`auth.store.ts`)

typescript

```
interface AuthState {
  user: User | null;
  accessToken: string | null;
  isAuthenticated: boolean;
  isLoading: boolean;

  // Actions
  setAuth: (user: User, token: string) => void;
  clearAuth: () => void;
  refreshToken: () => Promise<void>;
}
```

The `accessToken` lives in Zustand memory only — never `localStorage`. On mobile, the refresh token is stored in `expo-secure-store`.

### 4.3 Role-based rendering

All role-gating is a frontend concern. The backend enforces RBAC on every request; the frontend mirrors it for UX, not security.

```
typescript
// shared/types/index.ts
export type UserRole = 'farmer' | 'buyer' | 'transporter' | 'agent' | 'admin'

// Usage in components
const { user } = useAuth();
if (user?.role !== 'farmer') return <Redirect to="/unauthorized" />;
```

## 5. Page and screen map

### 5.1 Web — React Router routes

|        |                                             |
|--------|---------------------------------------------|
| /      | → redirect to /login or /dashboard based on |
| auth   |                                             |
| /login | → LoginPage                                 |

|                           |                                                          |
|---------------------------|----------------------------------------------------------|
| /register                 | → RegisterPage (step 1: phone + name + role)             |
| /register/verify          | → OTPVerifyPage (step 2: enter OTP + set PIN)            |
|                           |                                                          |
| /dashboard                | → DashboardPage (role-aware, different content per role) |
|                           |                                                          |
| # Farmer routes           |                                                          |
| /farmer/listings          | → MyListingsPage                                         |
| /farmer/listings/new      | → CreateListingPage                                      |
| /farmer/listings/:id/edit | → EditListingPage                                        |
| /farmer/orders            | → FarmerOrdersPage                                       |
| /farmer/orders/:id        | → OrderDetailPage (farmer view)                          |
| /farmer/transport/:id     | → TransportTrackingPage                                  |
|                           |                                                          |
| # Buyer routes            |                                                          |
| /marketplace              | → MarketplacePage (browse + filter + AgriScore)          |
| /marketplace/:id          | → ListingDetailPage                                      |
| /marketplace/:id/order    | → PlaceOrderPage                                         |
| /buyer/orders             | → BuyerOrdersPage                                        |
| /buyer/orders/:id         | → OrderDetailPage (buyer view)                           |
|                           |                                                          |
| # Transporter routes      |                                                          |
| /transporter/jobs         | → AvailableJobsPage                                      |
| /transporter/jobs/:id     | → JobDetailPage                                          |
| /transporter/active       | → ActiveDeliveryPage (real-time tracking)                |
|                           |                                                          |
| # Agent routes            |                                                          |
| /agent/clients            | → ClientListPage                                         |
| /agent/clients/new        | → RegisterClientPage                                     |
| /agent/orders/pending     | → PendingConfirmationsPage                               |
|                           |                                                          |
| # Shared                  |                                                          |
| /orders/:id/messages      | → OrderMessagesPage                                      |
| /profile                  | → ProfilePage                                            |
| /profile/edit             | → EditProfilePage                                        |
| /ratings/:orderId         | → SubmitRatingPage                                       |

## 5.2 Mobile — Expo Router structure

```
app/
├─ (auth)/
│   ├─ login.tsx
│   ├─ register.tsx
│   └─ verify.tsx
│
├─ (tabs)/                                # Bottom tab navigator (role-aware
tabs)                                     tabs)
│   ├─ _layout.tsx                        # Tab bar – different tabs per role
│   ├─ home.tsx                          # Dashboard / home feed
│   ├─ marketplace.tsx                   # Buyer: browse listings
│   ├─ listings.tsx                      # Farmer: my listings
│   ├─ jobs.tsx                          # Transporter: available jobs
│   ├─ orders.tsx                        # All roles: my orders
│   └─ profile.tsx
│
├─ listings/
│   ├─ [id].tsx                          # Listing detail
│   ├─ [id]/order.tsx                    # Place order
│   └─ new.tsx                           # Create listing
│
├─ orders/
│   ├─ [id].tsx                          # Order detail
│   └─ [id]/messages.tsx                 # Order chat
│
├─ transport/
│   └─ [id]/track.tsx                    # Live delivery tracking
│
└─ agent/
    ├─ clients.tsx
    └─ pending.tsx
```

| Role        | Tab 1 | Tab 2       | Tab 3   | Tab 4   |
|-------------|-------|-------------|---------|---------|
| farmer      | Home  | My Listings | Orders  | Profile |
| buyer       | Home  | Marketplace | Orders  | Profile |
| transporter | Home  | Jobs        | Active  | Profile |
| agent       | Home  | Clients     | Pending | Profile |
|             |       |             |         |         |

## 6. Component tree

### 6.1 Shared components (usable in both web and RN via platform files)

Platform-specific files use `.web.tsx` / `.native.tsx` suffixes where needed. Shared logic lives in `.tsx` with no platform APIs.

```

shared/components/
├─ ProduceListingCard/
|   ├─ ProduceListingCard.tsx      # Core logic + AgriScore display
|   ├─ FreshnessIndicator.tsx      # Freshness % + risk badge
|   └─ UrgentSaleBadge.tsx         # "Urgent Sale" overlay
|
├─ OrderStatusBadge/
|   └─ OrderStatusBadge.tsx        # Color-coded status pill
|
├─ AgriScoreDisplay/
|   └─ AgriScoreDisplay.tsx        # 0-100 score ring + breakdown tooltip
|
├─ TrustBadge/
|   └─ TrustBadge.tsx              # "Verified Farmer" / "Trusted Buyer"
badge
|
├─ ConfirmationModeIndicator/
|   └─ ConfirmationModeIndicator.tsx # Shows which confirmation mode is
active
|
└─ SMSCommandGuide/
    └─ SMSCommandGuide.tsx         # Context-aware guide for YES/NO, PAUSE,
etc.

```

## 6.2 Web-only components

```

web/components/
├─ layout/
|   ├─ AppShell.tsx                # Sidebar nav + top bar + main content
area
|   ├─ Sidebar.tsx                 # Role-aware nav links
|   └─ TopBar.tsx                  # Search, notifications, user avatar
|
├─ marketplace/
|   ├─ ListingGrid.tsx             # Responsive grid of
ProduceListingCards
|   └─ FilterPanel.tsx             # Sidebar: veg type, price range,

```

|                             |                                         |
|-----------------------------|-----------------------------------------|
| distance, packaging         |                                         |
| └─ MapView.tsx              | # Leaflet map with farmer pin markers   |
| └─ SortBar.tsx              | # AgriScore / Price / Distance /        |
| Freshness                   |                                         |
|                             |                                         |
| └─ listings/                |                                         |
| └─ ListingForm.tsx          | # Create/edit listing (shared form      |
| logic)                      |                                         |
| └─ ImageUploader.tsx        | # react-dropzone → POST                 |
| /listings/upload-image      |                                         |
| └─ PackagingPicker.tsx      | # GET /packaging/recommend → selectable |
| cards                       |                                         |
| └─ PreAuthSettings.tsx      | # Set auto_confirm_until_kg + price     |
| floor                       |                                         |
|                             |                                         |
| └─ orders/                  |                                         |
| └─ OrderTimeline.tsx        | # Visual status progression             |
| └─ OrderBreakdown.tsx       | # Produce + packaging + transport cost  |
| table                       |                                         |
| └─ NegotiationBubble.tsx    | # Counter-price proposal UI             |
| └─ ConfirmationPanel.tsx    | # Farmer confirm/decline + mode         |
| indicator                   |                                         |
|                             |                                         |
| └─ transport/               |                                         |
| └─ TransportRequestCard.tsx | # Open job card for transporters        |
| └─ DeliveryTracker.tsx      | # Live status + Leaflet map route       |
| └─ OTPEntryModal.tsx        | # Transporter enters buyer OTP at       |
| handoff                     |                                         |
|                             |                                         |
| └─ agent/                   |                                         |
| └─ ClientCard.tsx           | # Agent's registered farmer/buyer card  |
| └─ PendingOrderRow.tsx      | # Agent confirmation queue row          |
|                             |                                         |
| └─ common/                  |                                         |
| └─ PhoneInput.tsx           | # E.164 format input with Ghana (+233)  |
| default                     |                                         |
| └─ PINInput.tsx             | # 4-6 digit PIN entry (masked)          |
| └─ LocationPicker.tsx       | # Leaflet map click → {lat, lng}        |

|                     |                                      |
|---------------------|--------------------------------------|
| ─ EmptyState.tsx    | # Consistent empty screen with CTA   |
| ─ SkeletonCard.tsx  | # Loading skeleton for listing cards |
| ─ ErrorBoundary.tsx |                                      |

### 6.3 Mobile-only components

```
mobile/components/
|─ layout/
|  |─ ScreenHeader.tsx      # Native back button + title + right
  action
|  |─ BottomSheet.tsx      # Reusable bottom sheet for filters,
  details
|
|─ marketplace/
|  |─ ListingFlatList.tsx  # FlatList with pagination + pull-to-
  refresh
|  |─ MapScreen.tsx        # react-native-maps with custom markers
|
|─ listings/
|  |─ CameraImagePicker.tsx # expo-image-picker + compress before
  upload
|  |─ GPSLocationButton.tsx # expo-location → current GPS
  coordinates
|
|─ transport/
|  |─ LiveTrackingMap.tsx   # react-native-maps + Socket.io position
  updates
|
|─ common/
|  |─ OTPInput.tsx         # 6-box OTP entry (native keyboard)
|  |─ MoMoSelector.tsx     # MTN / Vodafone / AirtelTigo picker
```

## 7. State management

### 7.1 What lives where

| State                                     | Store                                   | Why                                                           |
|-------------------------------------------|-----------------------------------------|---------------------------------------------------------------|
| Authenticated user + access token         | Zustand <code>auth.store</code>         | Needed globally, persisted across navigation                  |
| Active order being placed (multi-step)    | Zustand <code>cart.store</code>         | Survives navigation between listing → order → payment screens |
| In-app toast notifications                | Zustand <code>notification.store</code> | Triggered from anywhere (API success/error)                   |
| Server data (listings, orders, transport) | TanStack Query                          | Caching, background refetch, deduplication                    |
| Form state                                | React Hook Form (local)                 | Not global — each form manages its own state                  |
| UI state (modals, panels open/closed)     | <code>useState</code> (local)           | No reason to globalise                                        |

### 7.2 Auth store

```
typescript
```

```
// packages/shared/stores/auth.store.ts
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

interface User {
  id: string;
  name: string;
  phone: string;
  role: UserRole;
  profile_photo_url: string | null;
  average_rating: number;
}

interface AuthStore {
  user: User | null;
  accessToken: string | null;
  setAuth: (user: User, token: string) => void;
  clearAuth: () => void;
}

export const useAuthStore = create<AuthStore>((set) => ({
  user: null,
  accessToken: null,
  setAuth: (user, accessToken) => set({ user, accessToken }),
  clearAuth: () => set({ user: null, accessToken: null }),
}));
```

### 7.3 Cart store (multi-step order placement)

typescript

```
// packages/shared/stores/cart.store.ts
interface OrderDraft {
  listingId: string;
  listingSnapshot: ProduceListing; // snapshot at order time
  quantityKg: number;
  packagingTypeId: string | null;
  deliveryAddress: string;
  deliveryLocation: { lat: number; lng: number } | null;
  transportCostEstimate: number;
}

interface CartStore {
  draft: OrderDraft | null;
  setDraft: (draft: OrderDraft) => void;
  updateDraft: (partial: Partial<OrderDraft>) => void;
  clearDraft: () => void;
}
```

## 7.4 TanStack Query key conventions

typescript

```
// Query keys – deterministic, cacheable
const queryKeys = {
  listings: {
    all: ['listings'] as const,
    list: (filters: ListingFilters) => ['listings', 'list', filters] as const,
    detail: (id: string) => ['listings', 'detail', id] as const,
  },
  orders: {
    all: ['orders'] as const,
    list: (status?: string) => ['orders', 'list', status] as const,
    detail: (id: string) => ['orders', 'detail', id] as const,
  },
  transport: {
    available: (lat: number, lng: number) => ['transport', 'available', lat, lng] as const,
    detail: (id: string) => ['transport', 'detail', id] as const,
  },
  packaging: {
    all: ['packaging'] as const,
    recommend: (veg: string) => ['packaging', 'recommend', veg] as const,
  },
};
```

---

## 8. API client

### 8.1 Axios instance with JWT interceptor

```
typescript
```

```
// packages/shared/api/client.ts
import axios from 'axios';
import { useAuthStore } from '../stores/auth.store';

export const apiClient = axios.create({
  baseURL: process.env.EXPO_PUBLIC_API_URL ?? import.meta.env.VITE_API_URL,
  withCredentials: true, // sends httpOnly refresh cookie automatically on w
  headers: { 'Content-Type': 'application/json' },
});

// Inject access token on every request
apiClient.interceptors.request.use((config) => {
  const token = useAuthStore.getState().accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Auto-refresh on 401
let isRefreshing = false;
let failedQueue: Array<{ resolve: Function; reject: Function }> = [];

apiClient.interceptors.response.use(
  (res) => res,
  async (error) => {
    const original = error.config;
    if (error.response?.status !== 401 || original._retry) {
      return Promise.reject(error);
    }
    if (isRefreshing) {
      return new Promise((resolve, reject) => {
        failedQueue.push({ resolve, reject });
      }).then((token) => {
        original.headers.Authorization = `Bearer ${token}`;
        return apiClient(original);
      });
    }
    original._retry = true;
```

```

isRefreshing = true;
try {
  const { data } = await apiClient.post('/auth/refresh');
  const newToken = data.data.access_token;
  useAuthStore.getState().setAuth(useAuthStore.getState().user!, newToken);
  failedQueue.forEach(({ resolve }) => resolve(newToken));
  original.headers.Authorization = `Bearer ${newToken}`;
  return apiClient(original);
} catch {
  failedQueue.forEach(({ reject }) => reject(error));
  useAuthStore.getState().clearAuth();
  // Redirect to login – handled by router guard
} finally {
  isRefreshing = false;
  failedQueue = [];
}
}
);

```

## 8.2 API module pattern

Each domain gets its own typed API module:

typescript

```

// packages/shared/api/listings.api.ts
import { apiClient } from './client';
import type { ProduceListing, ListingFilters, CreateListingDto } from '../types';

export const listingsApi = {
  browse: (filters: ListingFilters) =>
    apiClient.get<ApiResponse<PaginatedResponse<ProduceListing>>>('/listings', {
      filters,
    }),

  getById: (id: string) =>
    apiClient.get<ApiResponse<{ listing: ProduceListing }>>(`/listings/${id}`),

  create: (dto: CreateListingDto) =>
    apiClient.post<ApiResponse<{ listing: ProduceListing }>>('/listings', dto),

  update: (id: string, dto: Partial<CreateListingDto>) =>
    apiClient.patch<ApiResponse<{ listing: ProduceListing }>>(`/listings/${id}`, dto),

  uploadImage: (file: File | Blob) => {
    const form = new FormData();
    form.append('image', file);
    return apiClient.post<ApiResponse<{ url: string; thumbnail_url: string }>>(
      '/listings/upload-image', form,
      { headers: { 'Content-Type': 'multipart/form-data' } }
    );
  },

  update: (id: string, status: TransportStatus, deliveryOtp?: string) =>
    apiClient.patch(`/transport/${id}/status`, {
      status,
      ...(deliveryOtp ? { delivery_otp: deliveryOtp } : {}),
      // delivery_otp required only when status = 'delivered' - backend returns 400 otherwise
    }),

  setPreAuth: (id: string, dto: { auto_confirm_until_kg: number; auto_confirm_until_kg_expiry: string }) =>
    apiClient.patch(`/listings/${id}/preauth`, dto),

  getPreAuthStatus: (id: string) =>
    apiClient.get<ApiResponse<{ status: PreAuthStatus }>>(`/listings/${id}/preauth-status`),
};

```

```
apiClient.get(`/listings/${id}/preauth/status`),

getPackagingRecommendations: (vegetableType: string) =>
  apiClient.get('/packaging/recommend', { params: { vegetable_type: vegetableType } });
```

## 9. Key screen specifications

### 9.1 MarketplacePage / marketplace.tsx

**Purpose:** Primary buyer discovery surface. Browse, filter, and find produce listings.

**Data:** `GET /listings` with filter params, `GET /listings` with `lat`/`lng`/`radius_km` for nearby.

**State:**

```
typescript

const [filters, setFilters] = useState<ListingFilters>({
  vegetable_type: '',
  min_price_kg: undefined,
  max_price_kg: undefined,
  min_quantity_kg: undefined,
  lat: undefined,
  lng: undefined,
  radius_km: 15,
  page: 1,
  limit: 20,
});

const [view, setView] = useState<'grid' | 'map'>('grid');
```

**Unique features rendered on this screen:**

- `AgriScoreDisplay` — composite score on each listing card
- `FreshnessIndicator` — freshness % + risk badge
- `UrgentSaleBadge` — highlighted listings approaching spoilage

- Map view toggle — Leaflet (web) / react-native-maps (mobile) with farmer pins

**API calls:**

- `GET /listings?page=1&limit=20&[filters]` — paginated listing feed
  - `GET /listings?lat=X&lng=Y&radius_km=15` — nearby listings (requires geolocation permission)
  - Transporter job browsing uses `GET /transport/available?lat=X&lng=Y` — separate endpoint, not `/listings`
- 

**9.2 CreateListingPage / new.tsx**

**Purpose:** Farmer creates a produce listing. Agent can create on behalf of assigned farmers.

**Data:** `POST /listings`, `GET /packaging/recommend?vegetable_type=X`

**Form fields (React Hook Form + Zod):**

| Field                                     | Input type                                                  | Validation                                                                          |
|-------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>vegetable_type</code>               | Select (tomatoes, peppers, garden eggs, okra, leafy greens) | Required                                                                            |
| <code>quantity_kg</code>                  | Number                                                      | > 0                                                                                 |
| <code>price_per_kg_ghs</code>             | Number                                                      | > 0                                                                                 |
| <code>harvest_date</code>                 | Date picker                                                 | Today or future                                                                     |
| <code>images</code>                       | ImageUploader (multi)                                       | Max 8MB per image (enforced by backend), max 5 images (frontend cap), jpeg/png/webp |
| <code>location</code>                     | LocationPicker map                                          | Required                                                                            |
| <code>recommended_packaging_id</code>     | PackagingPicker cards                                       | Optional — auto-populated from recommendation                                       |
| <code>auto_confirm_until_kg</code>        | Number (collapsible section)                                | Optional pre-auth                                                                   |
| <code>auto_confirm_price_floor_ghs</code> | Number (collapsible section)                                | Optional pre-auth                                                                   |
| <code>farmer_id</code>                    | Hidden / Agent select                                       | Agent only: select from client list                                                 |

### Flow:

1. Fill form → validate with Zod schema
2. Upload images first → receive Cloudinary URLs → store in form state
3. Fetch packaging recommendation when `vegetable_type` changes
4. Submit → `POST /listings` → redirect to `/farmer/listings`

---

### 9.3 PlaceOrderPage / [id]/order.tsx

**Purpose:** Buyer places an order from a listing detail page.

**Data:** (POST /orders), (GET /listings/:id), (GET /packaging)

**Form fields:**

| Field               | Input type            | Notes                                   |
|---------------------|-----------------------|-----------------------------------------|
| (quantity_kg)       | Number slider + input | Max = listing available quantity        |
| (packaging_type_id) | Card picker           | Pre-selected = listing's recommendation |
| (delivery_address)  | Text                  | Free text                               |
| (delivery_location) | Map picker            | Optional GPS pin                        |

**Live cost breakdown (computed client-side, confirmed by backend):**

```
Produce subtotal = quantity_kg × listing.price_per_kg_ghs
Packaging cost   = selected_packaging.cost_per_unit_ghs × ceil(quantity_kg
/ capacity_kg)
Transport est.   = shown as "estimate – calculated on transport request"
Total           = produce + packaging
```

**Post-submit:** Show (OrderConfirmationScreen) explaining which confirmation mode applies to this farmer. Read (order.confirmation\_mode) from the response (not (order.status)) — the addendum adds this as a separate field:

- (sms\_reply) → "The farmer will receive an SMS to confirm your order. You'll be notified when they respond."
- (agent) → "The farmer's agent will review and confirm your order."
- (auto) or (trusted\_buyer) → "Your order has been automatically confirmed. Proceed to payment."

This is why `confirmation_mode` is listed as a required field in the backend handoff notes (section 15, point 2).

---

#### 9.4 OrderDetailPage / [id].tsx

**Purpose:** Full order lifecycle view. Shared between buyer and farmer; content differs by role.

**Sections (all roles see):**

- `OrderTimeline` — visual status progression
- `OrderBreakdown` — produce + packaging + transport cost table
- Farmer contact card / Buyer contact card (counterparty info)

**Farmer-only sections:**

- `ConfirmationPanel` — confirm / decline / negotiate buttons (visible when status = `pending` or `negotiating`)
- Pre-auth settings shortcut — "Set up auto-confirm to handle orders like this automatically"

**Buyer-only sections:**

- Payment CTA — "Pay now" button (visible when status = `confirmed`), links to `POST /payments/initialize` then Paystack redirect)
- Delivery tracking link (visible when transport request exists)

**Transporter-only sections:**

- Delivery tracking link — only if a transport request exists for this order. OTP entry is on the transport tracking screen, not here.
- 

#### 9.5 ActiveDeliveryPage / [id]/track.tsx

**Purpose:** Real-time delivery tracking for transporter and buyer.

**Data:** Socket.io subscription to transport status updates, `GET /transport/:id`

**Socket.io events to subscribe to:**

```
typescript

// Connect on mount, disconnect on unmount
socket.on('transport:status_updated', (data: { requestId: string; status: Tra
  queryClient.invalidateQueries({ queryKey: queryKeys.transport.detail(data.r
});
```

**Status buttons (transporter only):**

- `accepted` → `en_route` → `picked_up` → `in_transit` → submit `delivered` + OTP code → `delivered`

Note: the OTP modal appears when the transporter taps "Mark as delivered". They enter the buyer's 6-digit code, which is sent in the request body as `delivery_otp` on `PATCH /transport/:id/status`. The backend calls Arkesel verify server-side — the frontend never calls Arkesel directly.

---

## 9.6 PendingConfirmationsPage / pending.tsx (Agent only)

**Purpose:** Agent reviews and confirms or declines orders for all assigned farmers.

**Data:** `GET /agent/orders/pending`, `PATCH /orders/:id/confirm`, `PATCH /orders/:id/decline`

**Per-order row shows:**

- Farmer name + crop + quantity + buyer name + price
  - Time since order placed (highlight if > 1 hour old)
  - Confirm / Decline buttons → decline requires a reason string (min 10 chars)
- 

## 9.7 FarmerInsightsDashboard (bonus — Phase 4)

**Purpose:** Market intelligence for farmers. Data-driven selling decisions.

**Widgets:**

- Average price for farmer's vegetable type this week (aggregate from listing data)
  - Most demanded produce nearby (top vegetable types by active order count in radius)
  - Farmer's own listings: freshness scores trending over time
  - Recommended selling price vs. current listing price
- 

**10. Unique features — implementation plan****10.1 Freshness score**

Computed client-side (no extra API call needed — all inputs come from the listing):

```
typescript
```

```
// packages/shared/utils/freshness.ts
interface FreshnessResult {
  percentage: number;
  risk: 'low' | 'medium' | 'high';
  urgentSale: boolean;
  recommendation: string;
}

const SPOILAGE_DAYS: Record<string, number> = {
  tomatoes: 7,
  peppers: 14,
  garden_eggs: 10,
  okra: 5,
  leafy_greens: 3,
};

export function computeFreshness(
  harvestDate: string,
  vegetableType: string,
): FreshnessResult {
  const maxDays = SPOILAGE_DAYS[vegetableType] ?? 7;
  const daysSince = Math.floor(
    (Date.now() - new Date(harvestDate).getTime()) / 86_400_000
  );
  const percentage = Math.max(0, Math.round(((maxDays - daysSince) / maxDays) * 100));
  const risk = percentage > 60 ? 'low' : percentage > 30 ? 'medium' : 'high';
  const urgentSale = percentage <= 20;
  const recommendation =
    urgentSale ? 'Sell today - discount recommended'
      : risk === 'medium' ? `Sell within ${maxDays - daysSince} days for maximum profit`
      : 'Good freshness - list at full price';
  return { percentage, risk, urgentSale, recommendation };
}
```

## 10.2 AgriScore™

typescript

```
// packages/shared/utils/agriScore.ts
interface AgriScoreInput {
  freshnessPercentage: number; // 0-100
  distanceKm: number; // buyer to farmer
  pricePerKg: number; // listing price
  marketAvgPricePerKg: number; // from aggregated listing data
  farmerRating: number; // 0-5
}

export function computeAgriScore(input: AgriScoreInput): number {
  const freshScore = input.freshnessPercentage;
  const distScore = Math.max(0, 100 - (input.distanceKm / 50) * 100);
  const priceScore = Math.max(0, Math.min(100,
    100 - ((input.pricePerKg - input.marketAvgPricePerKg) / input.marketAvgPr
  ));
  const ratingScore = (input.farmerRating / 5) * 100;

  // Weighted composite
  return Math.round(
    freshScore * 0.35 +
    distScore * 0.25 +
    priceScore * 0.25 +
    ratingScore * 0.15
  );
}
```

### 10.3 Shared delivery suggestion

After a buyer places an order, check if other nearby orders exist heading to the same general destination:

```
typescript
```

```
// Prompt shown on OrderConfirmationScreen
// Backend: GET /transport/available?lat=X&lng=Y returns open requests
// If multiple listings in same pickup zone → show suggestion card

interface SharedDeliverySuggestion {
  partnerOrders: number;
  estimatedSaving: number; // GHS
  savingPercent: number;
}
```

The UI shows: *"2 other farmers near you are also delivering to Kejetia Market. Combine deliveries and save GHS 48 (35%)."*

---

## 11. Forms and validation

All forms use React Hook Form with Zod resolver. Validation schemas in `packages/shared/schemas/` are kept in sync with backend Zod schemas (same field names and rules).

typescript

```
// packages/shared/schemas/listing.schemas.ts
import { z } from 'zod';

export const createListingSchema = z.object({
  vegetable_type: z.enum(['tomatoes', 'peppers', 'garden_eggs', 'okra', 'leafy_green']),
  quantity_kg: z.number().positive(),
  price_per_kg_ghs: z.number().positive(),
  harvest_date: z.string().refine((d) => new Date(d) >= new Date(new Date().toLocaleDateString()), {
    message: 'Harvest date must be today or in the future',
  }),
  images: z.array(z.string().url()).max(5),
  location: z.object({ lat: z.number(), lng: z.number() }),
  recommended_packaging_id: z.string().uuid().optional(),
  auto_confirm_until_kg: z.number().positive().optional(),
  auto_confirm_price_floor_ghs: z.number().positive().optional(),
});

export type CreateListingDto = z.infer<typeof createListingSchema>;
```

## 12. Real-time and offline

### 12.1 Socket.io (delivery tracking)

Socket.io connection is established only on transport tracking screens, not globally.

typescript

```
// packages/shared/hooks/useRealtimeStatus.ts
import { useEffect } from 'react';
import { io, Socket } from 'socket.io-client';
import { useAuthStore } from '../stores/auth.store';

let socket: Socket | null = null;

export function useRealtimeStatus(transportRequestId: string, onUpdate: (stat
  const token = useAuthStore((s) => s.accessToken);

  useEffect(() => {
    socket = io(process.env.EXPO_PUBLIC_API_URL!, {
      auth: { token },
      transports: ['websocket'],
    });

    socket.emit('join:transport', transportRequestId);
    socket.on('transport:status_updated', (data) => {
      if (data.requestId === transportRequestId) onUpdate(data.status);
    });

    return () => {
      socket?.disconnect();
      socket = null;
    };
  }, [transportRequestId, token]);
}
```

## 12.2 Offline behaviour

| Scenario                      | Behaviour                                                                |
|-------------------------------|--------------------------------------------------------------------------|
| No network on marketplace     | Show cached TanStack Query data with "Last updated X minutes ago" banner |
| No network on order placement | Disable submit, show "No connection — check your network"                |

| Scenario                        | Behaviour                                                         |
|---------------------------------|-------------------------------------------------------------------|
| No network on delivery tracking | Show last known status, hide live map, show "Live updates paused" |
| Image upload fails              | Show retry button per image, don't block rest of form             |
| API call fails (non-auth)       | Toast error with retry action                                     |

### 13. Environment variables

#### Web (`.env`)

```
VITE_API_URL=https://vegelink-api.onrender.com/api/v1
VITE_SOCKET_URL=https://vegelink-api.onrender.com
```

#### Mobile (`app.config.ts`) / (`.env`)

```
EXPO_PUBLIC_API_URL=https://vegelink-api.onrender.com/api/v1
EXPO_PUBLIC_SOCKET_URL=https://vegelink-api.onrender.com
```

### 14. Testing plan

| Layer                    | Tool                         | Coverage target                                                                |
|--------------------------|------------------------------|--------------------------------------------------------------------------------|
| Unit — utility functions | Vitest (web) / Jest (mobile) | <code>computeFreshness</code> , <code>computeAgriScore</code> , shared schemas |
| Component tests          | React Testing Library        | Form validation, role-gated rendering, empty states                            |
| Hook tests               | renderHook                   | <code>useAuth</code> , <code>useListings</code> , <code>useOrders</code>       |

| Layer        | Tool       | Coverage target                                          |
|--------------|------------|----------------------------------------------------------|
| E2E — web    | Playwright | Full buyer journey: browse → order → pay                 |
| E2E — mobile | Detox      | Farmer journey: create listing → receive order → confirm |

### Critical test cases

- `computeFreshness` returns `urgentSale: true` at  $\leq 20\%$  freshness
- `computeAgriScore` returns a value between 0-100 for all valid inputs
- `PhoneInput` enforces E.164 format with `+233` prefix
- `PlaceOrderPage` quantity slider cannot exceed listing's `available_kg`
- `ConfirmationPanel` is only rendered when `user.role === 'farmer'` on `OrderDetailPage`
- Token refresh interceptor retries the original request with the new token
- `OrderStatusBadge` renders the correct colour for all 11 possible statuses: `pending`, `negotiating`, `confirmed`, `packed`, `in_transit`, `delivered`, `cancelled`, `pending_agent_confirmation`, `farmer_disputed`, `pending_sms_confirmation`, `cancelled_expired`

## 15. Handoff notes for backend team

### What the frontend expects from every API response

```
typescript
```

```
// Success
{ "success": true, "data": { ... } }

// Error
{ "success": false, "error": { "code": "ERROR_CODE", "message": "Human readab

// Paginated list (resource key matches the domain, e.g. listings, orders)
{ "success": true, "data": { "listings": [...], "meta": { "page", "limit", "t
```

### Specific requests from frontend

1. **GET /listings** response must include **freshness\_inputs** — **harvest\_date** and **vegetable\_type** must be top-level fields on each listing object (needed for client-side freshness calculation). ✅ Already in schema.
2. **POST /orders** response must include **confirmation\_mode** — the frontend needs to know which mode was activated (agent / auto / sms\_reply / trusted\_buyer) to show the correct post-order message to the buyer.
3. **GET /listings?lat&lng&radius\_km** needs **distance\_km** on each result — to display distance to buyer and feed into AgriScore calculation.
4. **GET /packaging/recommend** response needs a **reason** string per option — e.g. "Best for tomatoes – prevents bruising". Shown in **PackagingPicker** cards.
5. **CORS:** Allow **http://localhost:5173** (Vite dev) and **http://localhost:8081** (Expo dev) in development. Production origins: **https://vegelink.vercel.app**.
6. **Socket.io namespace:** Use **/transport** namespace for delivery status events. Event name: **transport:status\_updated** with payload **{ requestId: string; status: string }**.

## 16. Demo seed data

The backend team seeds the following users for the final demo. Frontend confirms these credentials work end-to-end:

| User         | Phone         | PIN  | Role        |
|--------------|---------------|------|-------------|
| Abena Mensah | +233244000001 | 1234 | farmer      |
| Kwame Asante | +233244000002 | 1234 | buyer       |
| Agent Kofi   | +233244000003 | 1234 | agent       |
| Yaw Boateng  | +233244000004 | 1234 | transporter |

**Demo journey:** Agent Kofi registers Abena → Abena lists 50kg tomatoes at GHS 4.50/kg → Kwame finds listing, orders 30kg → pre-auth auto-confirms → Kwame pays via Paystack sandbox → Yaw accepts transport job → status updated to delivered with OTP → both leave ratings.